Documentacion del codigo del sistema de vision para sumobot

Versión 1.0

Juan Jose Prado Neira

25 de octubre de 2026

Contenido

1. Arquitectura del sistema **1**

2. Referencia de API **2**
2.1. Modulo principal 2
2.2. Recepcion de video 3
2.3. Procesado de imagen 3
2.4. Toma de decision 4
2.5. Envio al robot 4

Esta documentacion describe el codigo Python del sistema de vision y control del sumobot. El proyecto esta organizado como una cadena de cuatro bloques: recepcion de video, procesado de imagen, toma de decision y envio de respuesta.

1 Arquitectura del sistema

El punto de entrada es `main.py`. Este fichero carga los cuatro bloques desde sus carpetas, lee los `config.json` y ejecuta el bucle de control:

1. `0_recibir_sumocam` recibe los frames JPEG enviados por la ESP32-CAM.
2. `1_procesar_imagen` detecta las lineas del carril y objetos en la escena.
3. `2_toma_decision` decide si seguir el carril, activar el protocolo de expulsion de objetos o atender una orden manual.
4. `3_enviar_respuesta` traduce la accion final a comandos UDP para el robot.

La interfaz visual de OpenCV muestra una vista principal y tres paneles de diagnostico: mascara, ROI y vista bird/debug. La configuracion se mantiene en JSON para poder ajustar umbrales, velocidades y tiempos sin cambiar codigo.

2 Referencia de API

2.1 Modulo principal

Punto de entrada del sistema de vision y control del sumobot.

El modulo carga dinamicamente los cuatro bloques del proyecto, lee sus ficheros de configuracion, recibe frames UDP de la ESP32-CAM, ejecuta el procesado de imagen, aplica la toma de decision y envia ordenes UDP al robot. Tambien construye la ventana de diagnostico de OpenCV y gestiona el modo automatico/manual.

class main.**DebugRecorder**(*enabled=False*, *output_dir=None*, *fps=20*)

Bases: object

Grabador opcional de flujos de depuracion.

Cuando esta activado, crea una carpeta por ejecucion y guarda videos MP4 para la vista principal, la mascara, la ROI y la vista bird/debug.

close()

Cierra todos los escritores de video abiertos.

write(*stream_name*, *frame*)

Escribe un frame en el video asociado a *stream_name*.

class main.**ManualLabelRecorder**(*debug_recorder*)

Bases: object

Registra etiquetas y decisiones por frame para analisis posterior.

close()

Cierra el CSV de etiquetas si fue creado.

write(*frame_idx*, *time_s*, *mode*, *key*, *manual_action*, *manual_delta*, *sent_action*, *sent_delta*, *proc_result*)

Anade una fila CSV con estado de teclado, vision y orden enviada.

main.**compose_dashboard**(*frame_main*, *proc_result*, *decision*, *movement_enabled*, *mode='auto'*, *manual_action=None*, *manual_delta=None*)

Compone la ventana 2x2 de diagnostico y estado de control.

main.**draw_overlay**(*frame*, *movement_enabled*, *proc_result*)

Dibuja sobre el frame principal el estado de movimiento y vision.

main.**draw_waiting_frame**(*width*, *height*, *movement_enabled*, *port*)

Construye la pantalla mostrada mientras no llega video UDP.

main.**empty_decision**(*movement_enabled*)

Crea una decision neutra con la forma esperada por el bucle principal.

main.**handle_manual_key**(*key*, *movement_enabled*, *manual_action*, *manual_delta*, *curve_delta*, *allow_motion_keys=True*, *hold_action=True*, *stop_when_blocked=False*)

Actualiza el estado manual a partir de una tecla de OpenCV.

main.**key_name**(*key*)

Convierte el codigo de tecla de OpenCV en una etiqueta legible.

main.**kwargs_for_ctor**(*ctor*, *cfg*)

Filtra un diccionario para quedarse solo con parametros de un constructor.

main.**load_json**(*path*)

Lee un fichero JSON de configuracion y valida que sea un objeto.

main.**load_symbol**(*module_path*, *symbol_name*)

Carga un simbolo Python desde un fichero sin exigir que sea paquete.

Parámetros

- **module_path** – Ruta al fichero Python que contiene el simbolo.
- **symbol_name** – Nombre de la clase, funcion o constante que se quiere recuperar.

Devuelve

El objeto Python solicitado.

main.**main**()

Ejecuta el bucle principal de recepcion, vision, decision y envio.

main.**manual_action_from_key**(*key*, *curve_delta*)

Traduce teclas del modo manual a acciones de alto nivel.

main.**mode_label**(*mode*)

Devuelve el nombre visible del modo de control actual.

main.**movement_label**(*enabled*)

Devuelve el texto corto usado para estado de movimiento.

main.**report_toggle**(*decision*, *sender*)

Informa de cambios de movimiento y manda parada al desactivar.

main.**send_decision**(*sender*, *decision*)

Envia la accion de una decision y devuelve lo realmente solicitado.

main.**show_waiting_window**(*cfg_procesar*, *cfg_recibir*, *decision_maker*)

Dibuja la pantalla de espera de video.

main.**write_debug_frames**(*debug_recorder*, *frame_main*, *proc_result*)

Guarda las cuatro vistas de debug del frame actual.

2.2 Recepcion de video

Recepcion de video UDP enviado por la ESP32-CAM.

El modulo reconstruye frames JPEG fragmentados en datagramas UDP mediante una cabecera ligera con identificador de frame, numero de paquete y tamano total. La clase mantiene un ultimo frame valido durante una pequena ventana de gracia para evitar parones de interfaz ante perdidas puntuales de paquetes.

class receiver.**UdpVideoReceiver**(*host*, *port*, *frame_timeout=0.2*, *rotation=None*)

Bases: `object`

Reconstruye imagenes OpenCV a partir de paquetes UDP fragmentados.

read()

Devuelve el siguiente frame BGR completo o `None` si no hay video.

2.3 Procesado de imagen

Procesado de imagen para navegacion entre lineas y deteccion de objetos.

El modulo recibe frames BGR de OpenCV, detecta lineas oscuras del carril, estima el centro navegable y localiza objetos oscuros que puedan invadir la trayectoria. El resultado es un diccionario estructurado con accion sugerida, confianza, offset, imagenes de depuracion y objetos detectados.

class processor.**ImageProcessor**(*width=800*, *roi_top=0.6*, *roi_bottom=1.0*, *contrast_threshold=18*, *absolute_value_threshold=120*, *min_area=1000*, *confidence_stop=0.18*, *object_detect_enabled=True*, *object_min_area=1200*, *object_dark_value_threshold=70*, *object_dark_value_threshold_max=95*, *use_perspective=True*, *src_norm=None*)

Bases: `object`

Pipeline de vision basado en umbrales, morfologia y contornos.

process(*frame*)

Procesa un frame y devuelve decision visual y vistas de diagnostico.

Parámetros

frame – Imagen BGR leida de la ESP32-CAM.

Devuelve

Diccionario con `action`, `steering_percent`, `confidence`, `offset_px`, `reason`, `objects` y frames de depuracion.

2.4 Toma de decision

Toma de decision para seguimiento de carril y expulsion de objetos.

La clase principal combina tres comportamientos: habilitacion manual de movimiento, seguimiento automatico de las lineas detectadas y una rutina reactiva para empujar un objeto fuera del carril y reincorporarse mediante marcha atras curva.

class decision.**DecisionMaker**(*start_enabled=False*, *conf_min=0.2*, *deadband_px=15*, *min_delta=1*, *max_delta=10*, *kp=0.06*, *ki=0.001*, *kd=0.001*, *box_eject_enabled=True*, *box_trigger_min_area=10000*, *box_approach_frames=70*, *box_push_frames=70*, *box_rejoin_back_frames=70*, *box_cooldown_frames=20*)

Bases: `object`

Controlador de alto nivel que decide la accion final por frame.

step(*key*, *proc_result=None*)

Procesa una tecla y el resultado de vision para producir una accion.

Parámetros

- **key** – Codigo de tecla devuelto por OpenCV.
- **proc_result** – Diccionario generado por `ImageProcessor.process`.

Devuelve

Diccionario con accion, delta, estado de movimiento y estado del protocolo de objeto.

2.5 Envio al robot

Envio de acciones de alto nivel al robot SUMO por UDP.

El modulo traduce acciones semanticas del controlador a comandos ASCII que entiende el firmware de motores. Tambien limita la frecuencia de envio y aplica una parada segura al cerrar el programa.

class sender.**RobotCommandSender**(*host*, *port*, *speed=25*, *turn_speed=30*, *interval=0.12*)

Bases: `object`

Cliente UDP para enviar ordenes de movimiento al robot.

close()

Cierra el socket UDP.

send_action(*action*, *delta=None*)

Convierte una accion de alto nivel en el comando UDP correspondiente.

send_command(*command*, *force=False*, *response_timeout=None*)

Envia un comando ASCII crudo respetando antirrebote temporal.

Parámetros

- **command** – Texto exacto que recibira el firmware.
- **force** – Si es `True`, ignora repeticion y periodo minimo.
- **response_timeout** – Tiempo opcional para esperar una respuesta UDP.

Devuelve

Respuesta textual del robot o `None` si no llega ninguna.

stop()

Envia varias ordenes de parada para dejar el robot en estado seguro.

sender.**clip_percent**(*value*)

Limita una velocidad o correccion al rango aceptado por el firmware.